



Smart-Clearance

GOOGLE AI HACKATHON 2026 · RETAIL & COMMERCE · TECHNICAL ARCHITECTURE

Every near-expiry carton gets a second chance, routed by agents on Google Cloud.

Smart-Clearance finds an FMCG brand's stock that will not sell before its date, reads the carton's label, prices every way out, and waits for one person's yes. Then agents list the lot, offer it to kiranas, answer buyers, draft the GST papers and post the impact ledger. This page shows how it is built: Firebase at the edge, Cloud Run for the API and the agents, Gemini on Vertex AI for the words, Pub/Sub for the events, Cloud SQL as the system of record and BigQuery for the analytics.

● Landing page

● Workspace app

● Staff console

● Guided demo

Source on GitHub

Source: github.com/aibuilder2026/smart-clearance · Live: smartclearance.web.app, munchly-smartclearance.web.app, smartclearance-console.web.app, smartclearance-demo.web.app. Every company, person and figure in Smart-Clearance is fictional; Munchly Foods is the demo client.

The stack at a glance

☐ Google Cloud ☐ Our code ☒ AI

EXPERIENCE

SvelteKit apps

Firebase Hosting

Firebase Auth

Cloud Messaging

SERVICES

backend-api · FastAPI

Cloud Run

Cloud Run jobs

INTELLIGENCE

Gemini on Vertex AI

Agent Development Kit

agents service

EVENTS

Pub/Sub

Cloud Scheduler

transactional outbox

SSE stream

DATA

Cloud SQL · PostgreSQL

BigQuery

Cloud Storage

TRUST

Secret Manager

IAM, no keys

Workload Identity Federation

DELIVERY

GitHub Actions

Cloud Build

Artifact Registry

Terraform

OPERATIONS

Cloud Logging

Cloud Trace

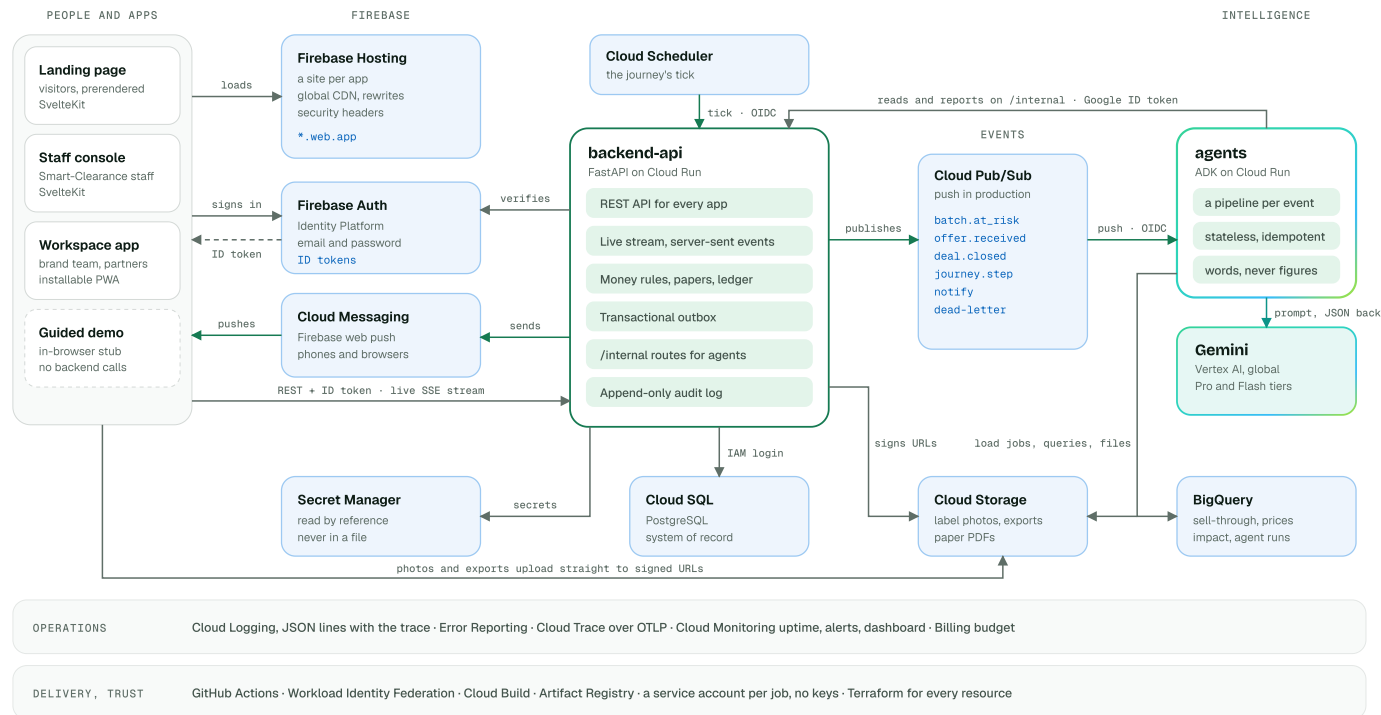
Cloud Monitoring

Error Reporting

THE SYSTEM

One picture of everything that runs

Static apps on Firebase Hosting call one API on Cloud Run. The API is the only writer: every change commits in one transaction with its audit line and its outgoing events, then Pub/Sub carries the events to a stateless agents service that asks Gemini for words and reports back. Pushes reach phones through Firebase Cloud Messaging; screens move on over a live stream.



Blue boxes are managed Google Cloud and Firebase services; green outlines are Smart-Clearance's own code; the gradient outline is AI. Green arrows are events and pushes, grey arrows are requests, dashed arrows are replies.

Google Cloud Our code AI

Firebase, Cloud Run and Gemini, and why Firestore is not here

Each Google product was picked for a job the product has to do. Where one would have been a poor fit, we said so and used the one that fits.



Firebase

Hosting, Authentication, Cloud Messaging, web apps

- **Hosting** serves each SvelteKit app as a static build on its own site: the prerendered landing page, the staff console, Munchly's workspace PWA and the guided demo. Deep links rewrite to the app, hashed assets cache as immutable, and the private apps send `noindex`, frame-denial and referrer headers.
- **Authentication** (Identity Platform) signs everyone in with email and password from one user pool. Sign-up is off, so only backend-api creates accounts; a password policy and email-enumeration protection are on. The API verifies each ID token with `firebase-admin`.
- **Cloud Messaging** carries the Notifier's pushes to members' phones and browsers through the PWA's service worker, so a distributor hears about a label photo request and an approver about a plan waiting.
- **Web apps and restricted keys:** each app has its Firebase web app config, written by Terraform, and a browser key limited to the identity APIs from that app's own origins.

WHY The apps are static over an API, so the whole frontend lives on Google's CDN with no server to run, and identity and push come from the same project with no password store or push service of our own.



Cloud Run

Two services, two jobs

- **backend-api**, FastAPI on Python, is the public API and the system's only writer. It serves the REST routes, the live stream, the agents' `/internal` routes and the tick.
- **agents**, Google ADK on Python, takes internal traffic only. Cloud Run's IAM lets in one identity, `sc-invoker`, which Pub/Sub's push subscriptions sign as.
- **Jobs** on the same image: `backend-api-migrate` runs Alembic as the schema owner, and `backend-api-hydrate` builds a synthetic world through the services. Cloud Build runs the migration before every deploy.
- Each runs as its own service account with an IAM database login, writes JSON logs that carry the trace, scales to zero, and rolls out one revision at a time behind a readiness check.

WHY Containers without servers: the same image runs on a laptop and in the cloud, service-to-service calls are signed with OIDC instead of shared secrets, and a quiet prototype costs next to nothing.



Gemini on Vertex AI

Two tiers, through the Agent Development Kit

- **Gemini Pro** (`gemini-3.1-pro-preview`) writes where judgement shows: the Valuer's note on each exit, the Router's explanation of the split, the Negotiator's reply to a bid, and the evals' judge.
- **Gemini Flash** (`gemini-3.8-flash`) reads and drafts: Vision reads the printed label from a distributor's photo, the Data agent maps an unfamiliar DMS export, and the Lister, Outreach (offers in Hindi, English and Marathi), Paperwork and Impact write their copy.
- Called through `google-genai` on Vertex AI's global endpoint with the service account's own credentials: no API key exists. Every call asks for structured output against a schema.
- Gemini supplies words only. backend-api computes every rupee, the words are checked against those figures, and a template stands in for any reply that fails.

WHY A multimodal model reads a crumpled carton label as printed, writes fluent shop offers in Indian languages, and runs under the project's own IAM, quotas and audit logs.



Firestore

Not used, by design

Cloud SQL for PostgreSQL holds the record instead

- The workspace is a ledger: money worked out by one set of rules, papers numbered in series, credit notes per line, and an audit log nobody may edit. That needs transactions across tables, constraints, triggers and SQL.
- **Live updates**, the usual reason to reach for Firestore, come from PostgreSQL itself: a change NOTIFYs as it commits, and members read it as server-sent events, cut to their role on the server.
- **Events leave only after commit**: each change writes its Pub/Sub messages to an outbox in the same transaction, so an agent never acts on a change that rolled back.
- The same PostgreSQL runs on a laptop in Docker, so every test runs against a real database, never a mock.

WHY Firebase does what it is best at here (hosting, sign-in and push), and the record lives where transactions, joins and an append-only audit log come built in.

What each service does here, and why we chose it

All of it lives in one Google Cloud project, written in Terraform. Nothing is made by hand in a console, and no service account has a key.

Edge and identity

Where people meet the product



Firebase Hosting

A site per app, released by `firebase-tools`: the landing page prerendered with its data, and the console, workspace and demo as single-page apps with deep-link rewrites and immutable caching for hashed assets.

WHY Static builds on a global CDN, each app with its own address, release and rollback.



Firebase Authentication

Identity Platform with email and password only, sign-up disabled, a password policy and email-enumeration protection. backend-api verifies ID tokens and makes accounts under a custom role limited to user management.

WHY Managed sign-in with standard ID tokens, and no password store of our own.



Firebase Cloud Messaging

The Notifier sends each notification to the member's registered devices through the workspace PWA's service worker. backend-api holds a custom role that may only send messages.

WHY One push channel for Android, installed iOS web apps and desktop browsers.

Compute

Containers, no servers



Cloud Run services

`backend-api` takes public traffic from the apps. `agents` takes internal traffic only, and only from `sc-invoker`, the identity Pub/Sub signs its pushes with.

WHY Scale to zero, a revision per release, service-to-service auth with OIDC tokens.



Cloud Run jobs

`backend-api-migrate` runs Alembic as the schema owner's IAM login; `backend-api-hydrate` builds the synthetic world through the services. Both use the API's image.

WHY One-off work with the same image and identity model, and no bastion host.

Intelligence

Gemini writes; the rules decide



Vertex AI, Gemini

Pro for the Valuer, Router, Negotiator's bids and the evals' judge; Flash for Vision's multimodal label read, the Data agent's column mapping, the Lister, Outreach, Paperwork and Impact. Global endpoint, structured output, retries with backoff on quota and server errors.

WHY Multimodal reading, Indian-language copy, and enterprise IAM and quotas in the same project.



Agent Development Kit

Each event runs one pipeline:

`SequentialAgent` where a step needs the last one's result, `ParallelAgent` where it does not, `LlmAgent` with an `output_schema` for each model call, and custom steps for the deterministic work.

WHY Google's framework for Gemini agents: composable, traced, and testable on recorded responses.

Events and time

Decoupling a tap from the work



Pub/Sub

Topics per environment for the agents' events and the Notifier, ordered by batch, with dead-letter topics. Production pushes to Cloud Run with an OIDC token; a laptop pulls the local subscriptions. Each message carries its `traceparent`.

WHY At-least-once delivery with retries, ordering that keeps a batch's steps in sequence, and the same code locally with no emulator.



Cloud Scheduler

The journey's clock: `sc-tick` calls backend-api's tick every minute with an OIDC token. It runs the daily agents on the journey's own time, fires due timers, re-sends stalled events and flushes the outbox.

WHY A managed clock with the same keyless auth, so a compressed demo day and a real one run on one mechanism.

Data

Record, analytics, files



Cloud SQL for PostgreSQL 18

The system of record: clients, members, batches, cases, papers, the ledger and the audit log. IAM logins only, through the connector, over TLS; the app's login has data access only, the audit log refuses edits by trigger; backups and point-in-time recovery.

WHY A ledger needs transactions and constraints, and LISTEN/NOTIFY gives the live stream for free.



BigQuery

The agents' analytics: stock snapshots and secondary sales loaded from DMS exports, channel prices, the impact ledger, one row per agent run (model, latency, fallback, trace), and the evals' results.

WHY Sell-through across distributors and pincodes is an analytical query; load jobs never load a file twice.



Cloud Storage

Buckets for label photos, DMS exports and Paperwork's PDFs, per environment. Phones upload straight to short-lived signed URLs, and papers open through signed links. Build sources and Terraform state live here too.

WHY Files never pass through the API, and lifecycle rules tidy what is no longer needed.

Trust

No keys, no secrets in files



Secret Manager

The few secrets there are, such as the default password new accounts start on. Scripts generate them and pipe them in on stdin; services read them by reference at runtime.

WHY Nothing secret in a file, a command line, a log, Terraform state or git.



IAM and service accounts

An account per job: the API, the agents, the migrator, the invoker, the builder and two CI deployers. Developers impersonate a local account in code from their own credentials. Custom roles for user management and messaging.

WHY Least privilege, and every action in the audit logs under the identity that did it.



Workload Identity Federation

GitHub Actions trades its OIDC token for short-lived Google credentials, accepted only from this repository's ids and the protected `prod` environment, which only `main` may deploy to.

WHY CI deploys with no stored credential at all.

Delivery

From a merge to a revision



Cloud Build

Builds each image with the last one's layers as cache, pushes it, runs the migration job on it, deploys the service and checks it answers ready. CI starts it as `sc-builder`, an account that may do only that.

WHY Builds run inside Google's network, next to the registry and Cloud Run.



Artifact Registry

A private Docker repository for the API's and the agents' images, each tagged with its commit, with a cleanup policy. Cloud Build owns the images; Terraform ignores them.

WHY In-region, private, and every running revision traceable to a commit.

Operations

Seeing a request end to end



Cloud Logging and Error Reporting

Every service writes one JSON object a line with severity, source and the request's trace, so Logs Explorer groups a request's lines and Error Reporting groups stack traces.

WHY One place to read a request, from either service.



Cloud Trace, Telemetry API

OpenTelemetry spans over OTLP: each route, SQL statement and token check in the API, and each ADK agent, Gemini call and BigQuery job in the agents, all under the trace the browser started.

WHY Google's recommended route into Cloud Trace, joined to Cloud Run's own sampled requests.



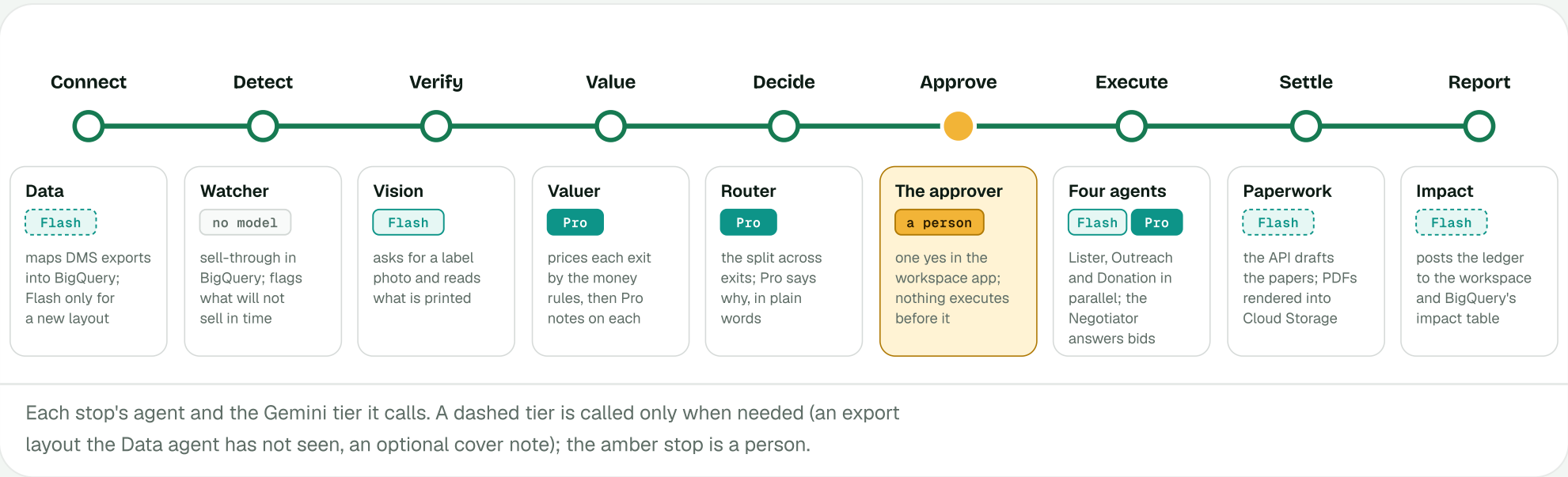
Cloud Monitoring and budgets

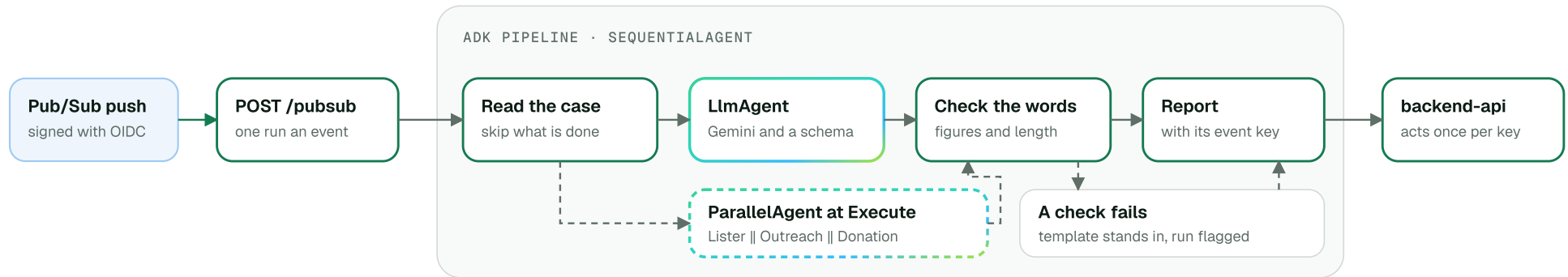
An uptime check on the API's readiness, alert policies for downtime, errors, latency, the database, dead letters, agent errors and fallbacks and push failures, log-based metrics, a dashboard and a billing budget.

WHY Alerts reach the operator by email before a person notices.

Ten agents along nine stops, and one person's yes

A batch at risk moves through the same stops every time. Agents do the reading, the writing and the chasing; backend-api does the arithmetic; and nothing leaves the godown until the approver says yes.





Inside one run. A redelivered event reads the case first and resumes where the last run stopped; backend-api answers a repeated event key with a no-op, so nothing happens twice.

Figures are computed, never generated

backend-api works out every price, split, credit note and ledger line by one set of money rules. The agents read previews of those figures and write words around them.

The reserve never reaches a prompt

The lowest price a buyer may get is cut from every case and preview an agent reads, and tests try hostile buyer messages to prove it.

A buyer's words are data

Messages go into the Negotiator's prompt between delimiters they cannot forge, as quoted text, never as instructions.

Every word is checked

Each figure in the copy must be one of the computed ones, a reply to a bid states the decided price and nothing else, and offers keep their shop placeholder and push length.

Structured output Gemini can fill

Every schema field is required, with null meaning absent, and no schema holds a free-keyed map, after the first live eval run showed how those come back.

Bounded calls

Each model call has a deadline, retries quota and server errors with backoff, and counts against a per-run budget; past it, the template stands in.

A batch's journey, request by request

Pick a stage to see which component calls which, in order. The core loop is the same at every stop: a person or a clock acts, backend-api commits, Pub/Sub carries the event, an agent works and reports, and every member's screen moves on.

The pages that follow take each stage in turn: the core loop and signing in first, then a batch's journey from Connect to Report.

- | | |
|-------------------------|---|
| 1. The core loop | 7. Decide · Router |
| 2. Signing in | 8. Approve · a person |
| 3. Connect · Data agent | 9. Execute · Lister, Outreach, Donation, Negotiator |
| 4. Detect · Watcher | 10. Settle · Paperwork |
| 5. Verify · Vision | 11. Report · Impact |
| 6. Value · Valuer | |

Solid grey: a request **Dashed grey:** a reply **Dashed green:** an event, push or stream **Teal:** a Gemini call **Amber:** a person's yes

The core loop

Every stop of the journey runs on this loop. A person acts in an app or the clock ticks; backend-api checks who it is, commits the change with its audit line and its outgoing events in one transaction, streams the change to every member, and publishes the events once the commit has landed. An agent picks the event up, gathers facts, asks Gemini for words, and reports back; the report is itself a change, so the loop turns again.

Firebase Authentication

Cloud Run

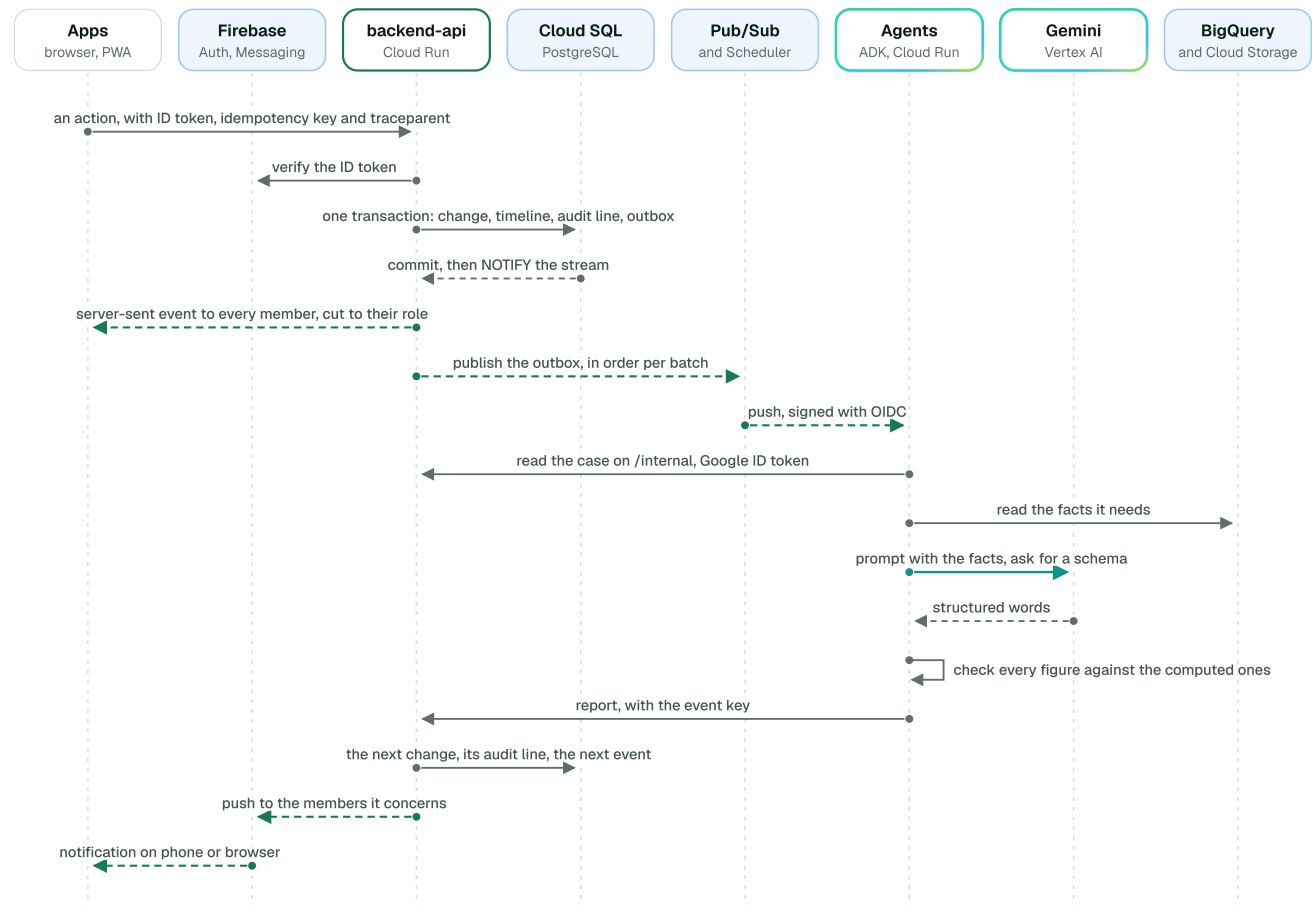
Cloud SQL

Pub/Sub

Vertex AI

BigQuery

Cloud Messaging



Each report names the event it answers. A redelivered event finds its work done, and backend-api answers the repeat with a no-op.

Signing in

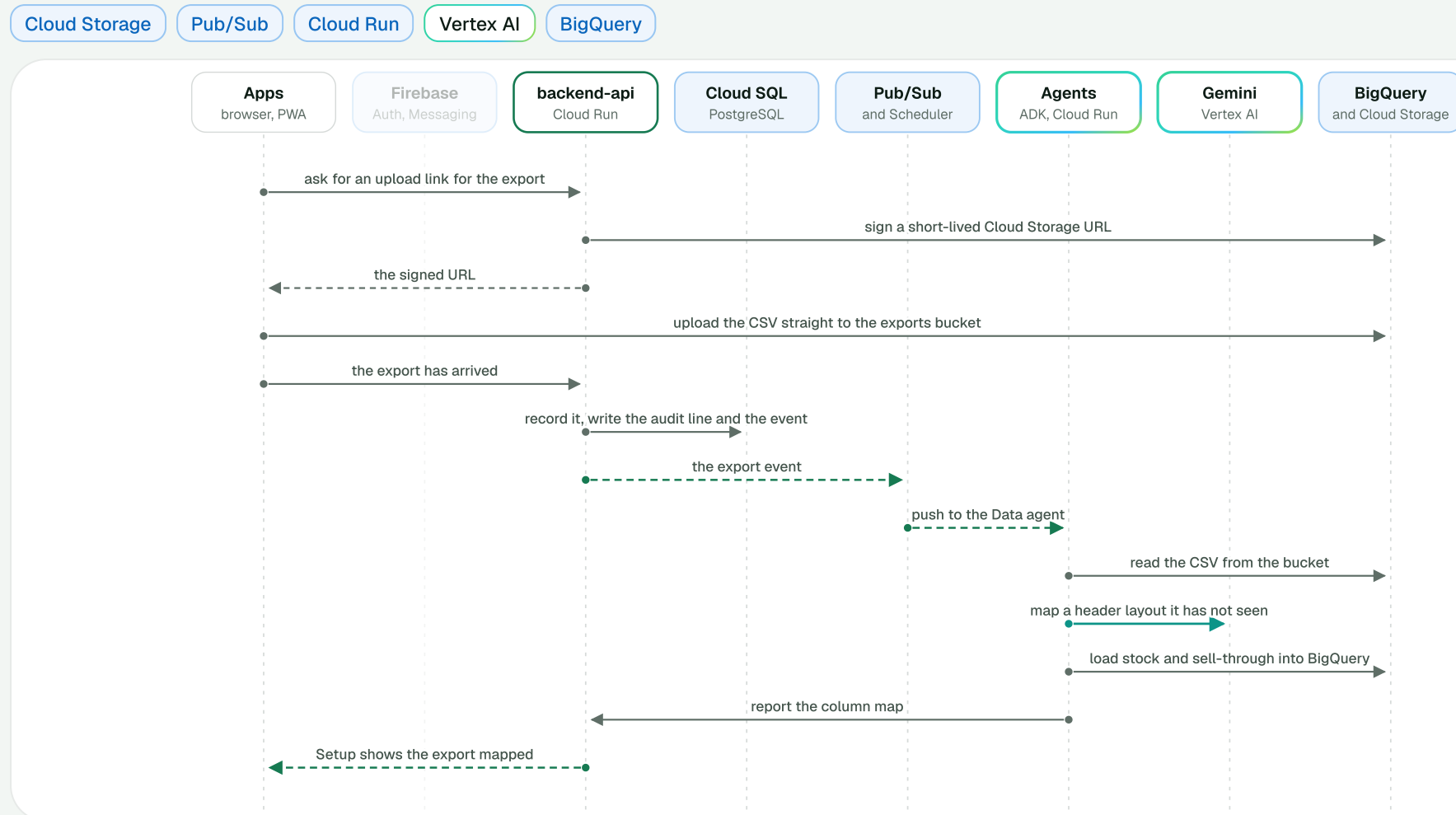
Everyone signs in with email and password through Firebase Authentication. The browser holds the ID token, the Firebase SDK refreshes it, and every call to backend-api carries it. The API verifies the token, finds the active member and their role, and from then on every answer is cut to what that role may see.



Sign-up is disabled, so only backend-api creates accounts. Email enumeration protection gives every wrong sign-in the same answer. A Firebase account that is not a member of this workspace is signed out again.

Connect • Data agent

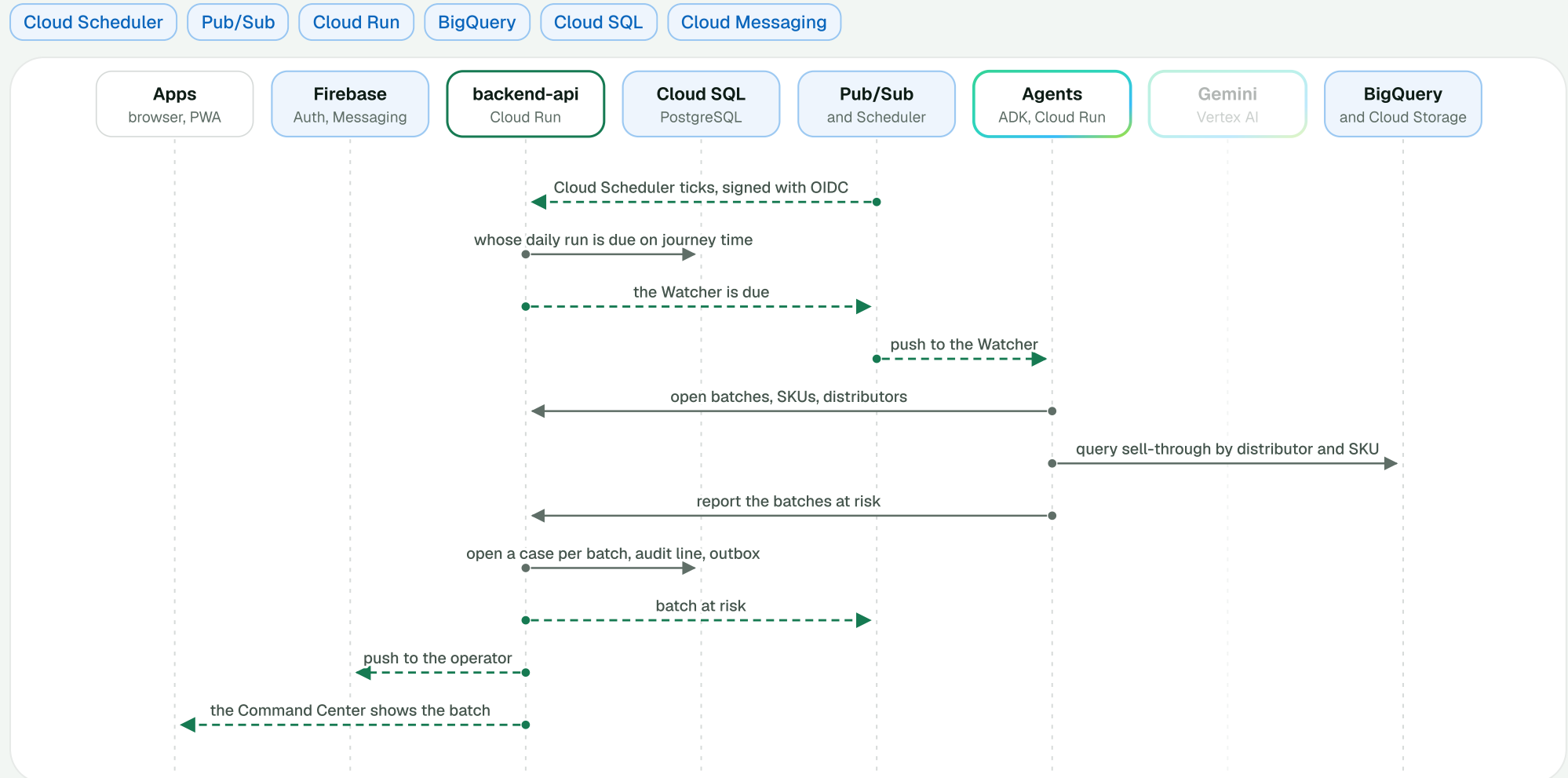
The brand's distributor management system exports stock and secondary sales as CSV. The file goes straight from the browser to Cloud Storage on a signed URL, never through the API. The Data agent reads it, maps its columns (with a saved layout, or Gemini Flash for headers it has not seen), and loads it into BigQuery with a load job.



A load job never loads the same file twice. Staff can map a client's first export from the console, so the workspace opens already connected.

Detect · Watcher

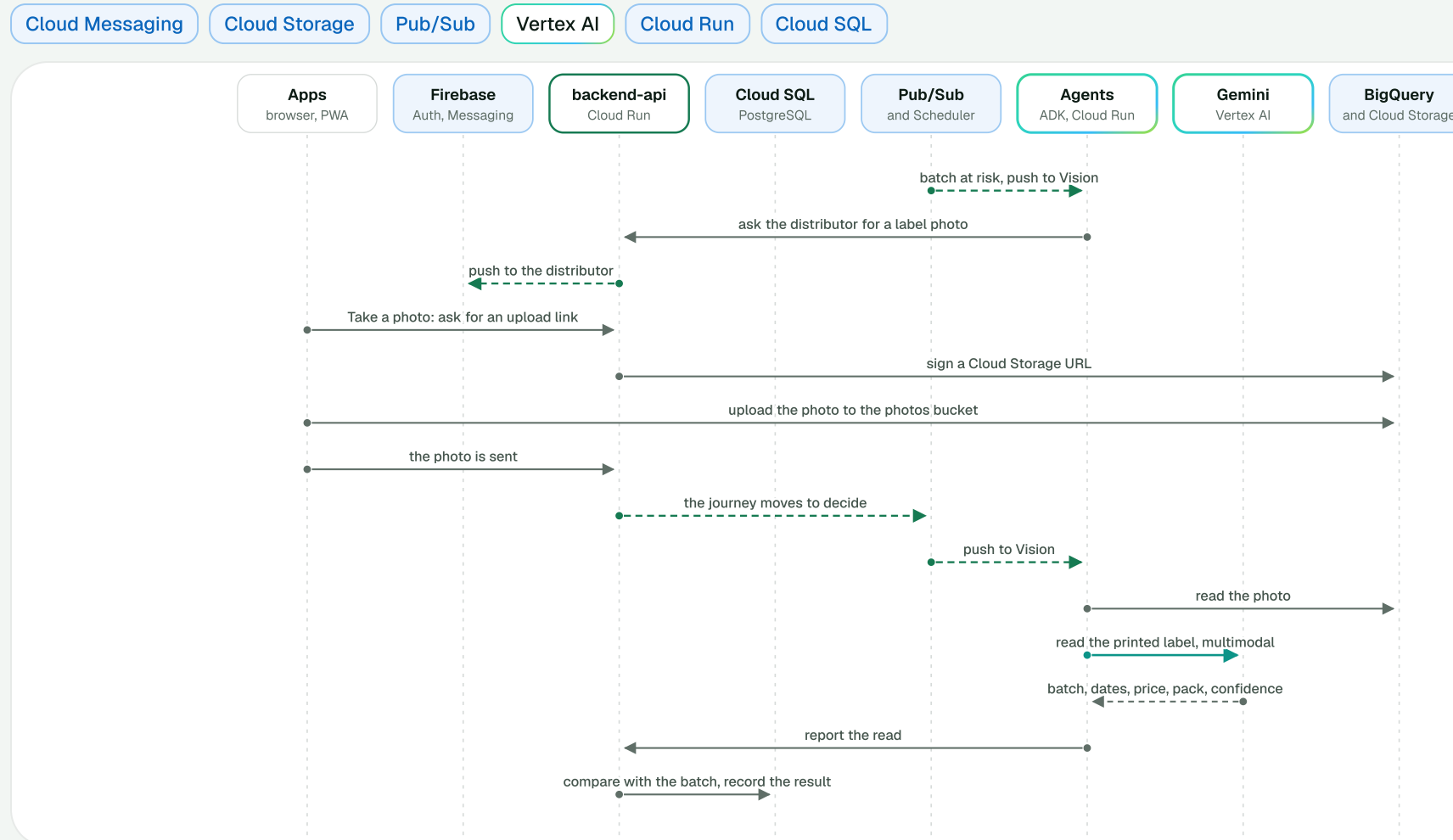
Cloud Scheduler ticks backend-api, which runs each client's daily jobs on the journey's own clock. The Watcher reads each open batch's sell-through from BigQuery and reports which batches will not sell before retailers stop taking them. backend-api opens a case for each and tells the operator.



The Watcher needs no model: sell-through is a query. It flags a batch only once Setup is confirmed and the distributor has given its permission.

Verify · Vision

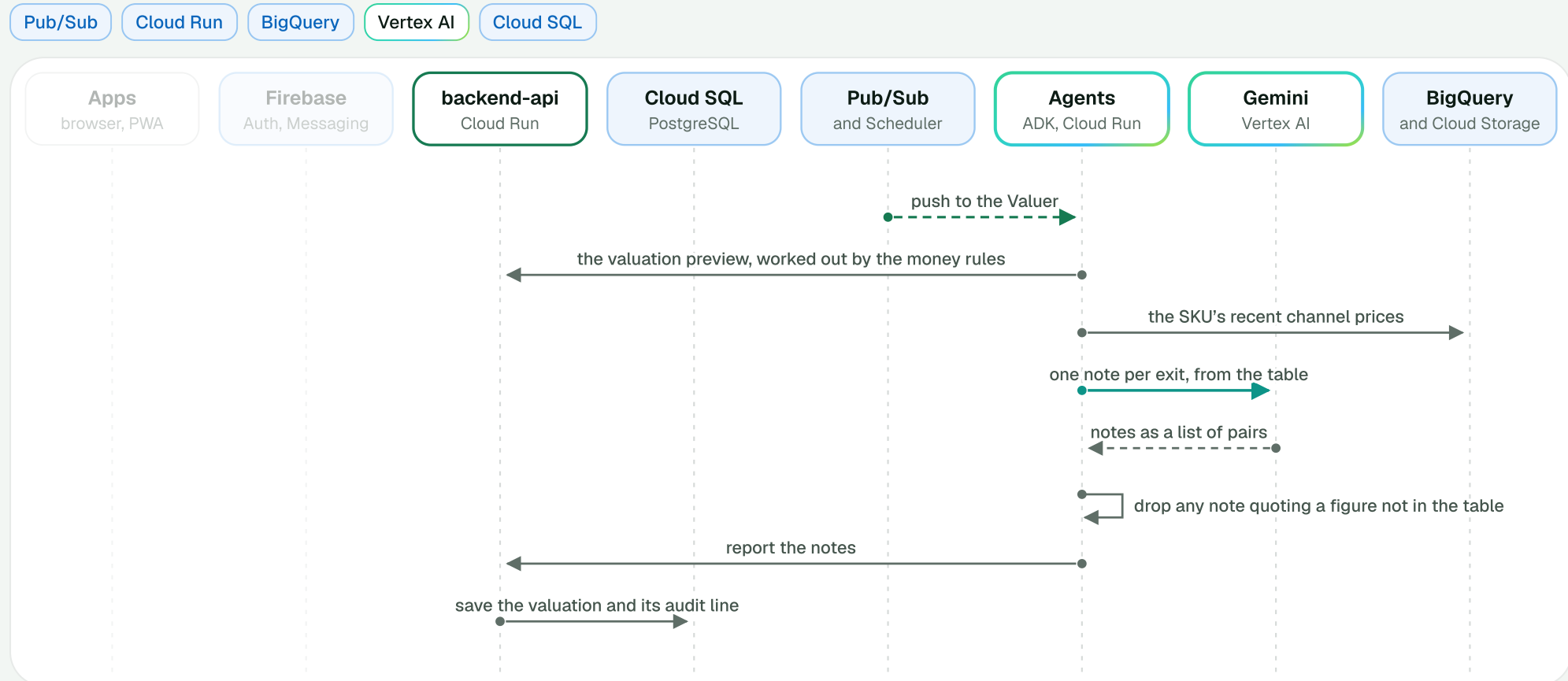
Before anything is priced, the batch is checked on the shelf. Vision asks the distributor for one photo of the carton's label; the phone uploads it straight to Cloud Storage; Gemini Flash reads what is printed, without being shown the record it is checked against. backend-api compares the read with the batch.



A label that cannot be read, or reads differently from the record, asks for another photo with the reason. A read Vision could not make is retried by Pub/Sub before it gives up.

Value • Valuer

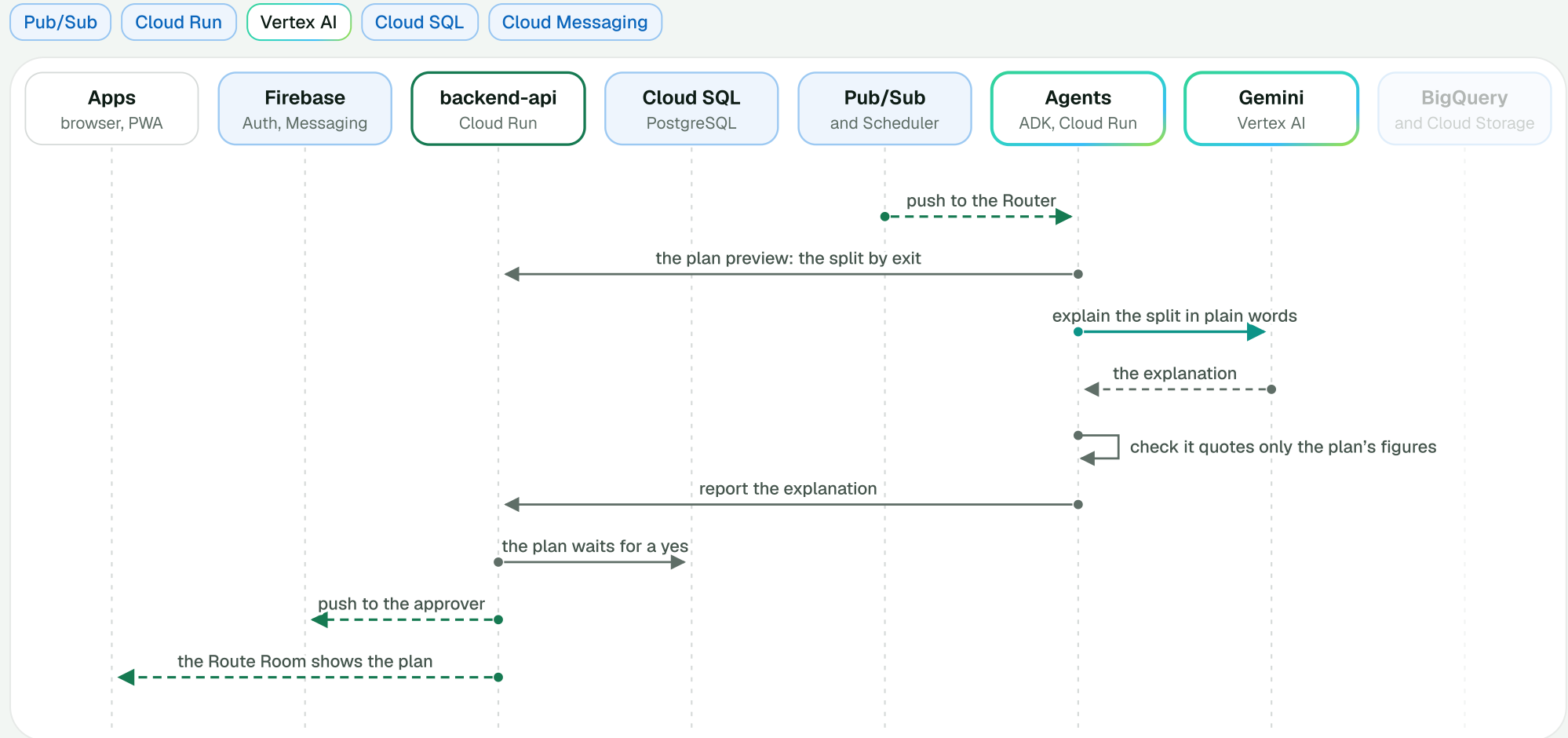
backend-api prices every way out for the batch with the money rules: kiranas, the quick-commerce marketplace, a staff sale, a food bank, or the bin. The Valuer reads that table and the SKU's recent prices in BigQuery, and Gemini Pro writes one note per exit saying why it suits this batch now.



The table comes from backend-api, never from the model. A note that quotes a figure not in the table is dropped, and the template stands in.

Decide · Router

backend-api splits the batch across the exits by the client's gates and guardrails. The Router reads the plan and Gemini Pro explains it in plain words for the approver, quoting only the plan's own figures. The plan then waits for a person.



If the explanation fails a check, the approver still sees the plan, explained by the template.

Approve • a person

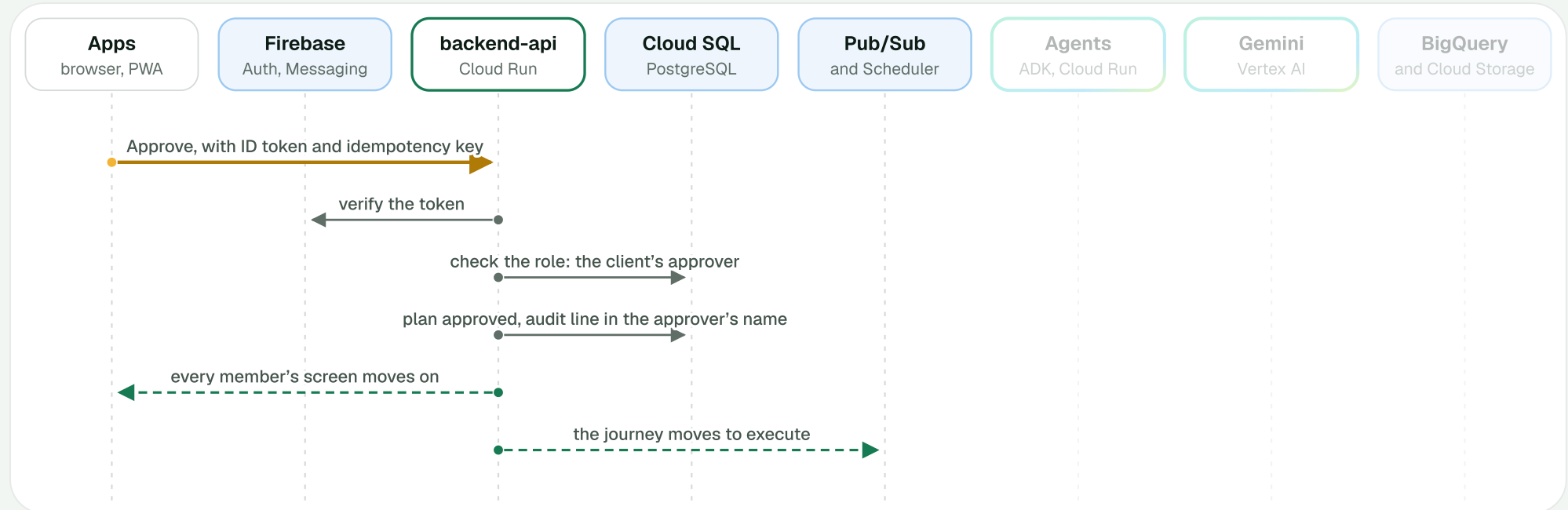
The approval is a person, never an agent. The approver reads the plan, the notes and the explanation, and says yes in the workspace app. backend-api checks the role, records the yes in the approver's name, and only then sends the event that starts execution.

Firebase Authentication

Cloud Run

Cloud SQL

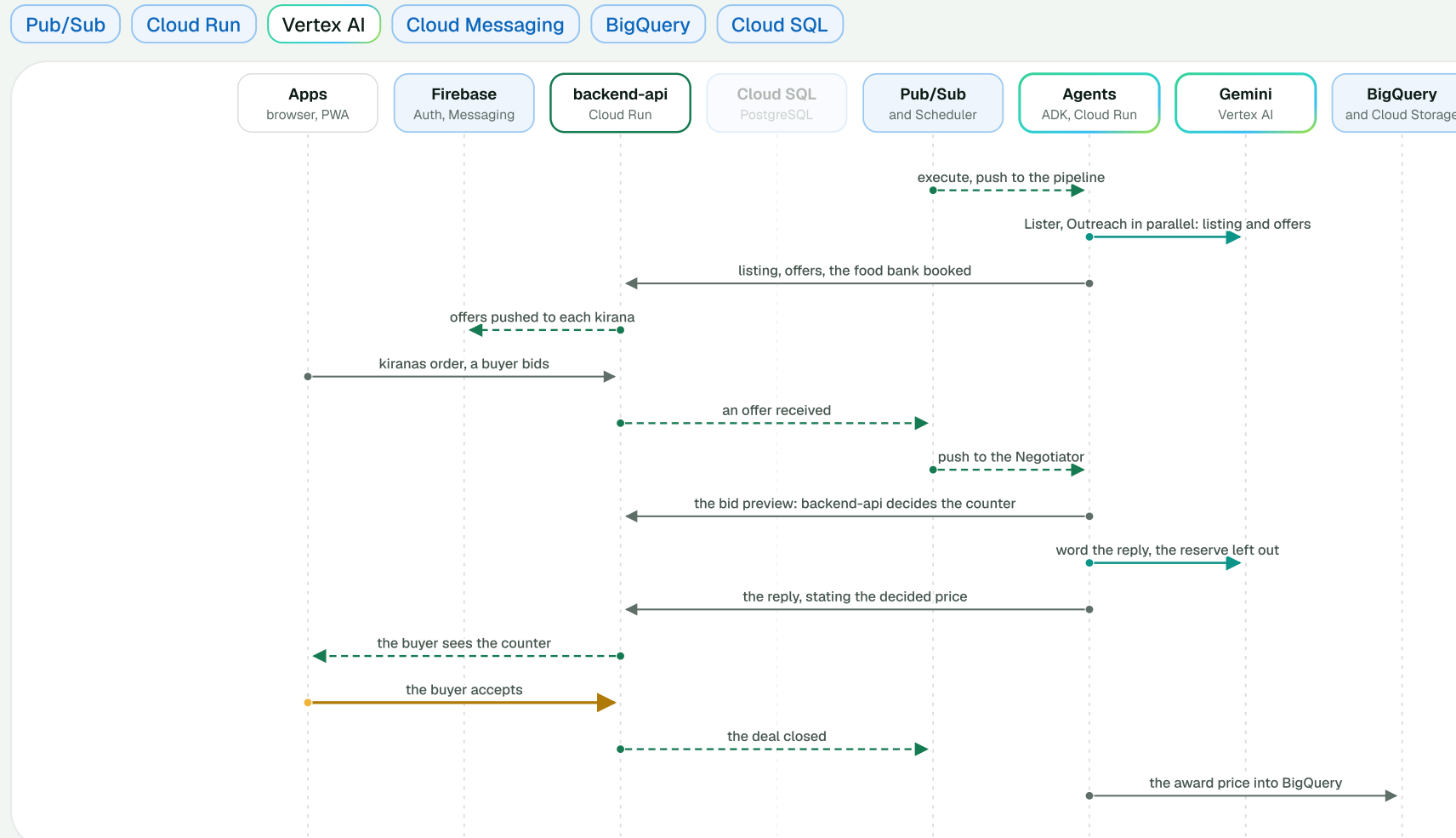
Pub/Sub



The yes carries an idempotency key, so a second tap on a slow network approves once.

Execute · Lister, Outreach, Donation, Negotiator

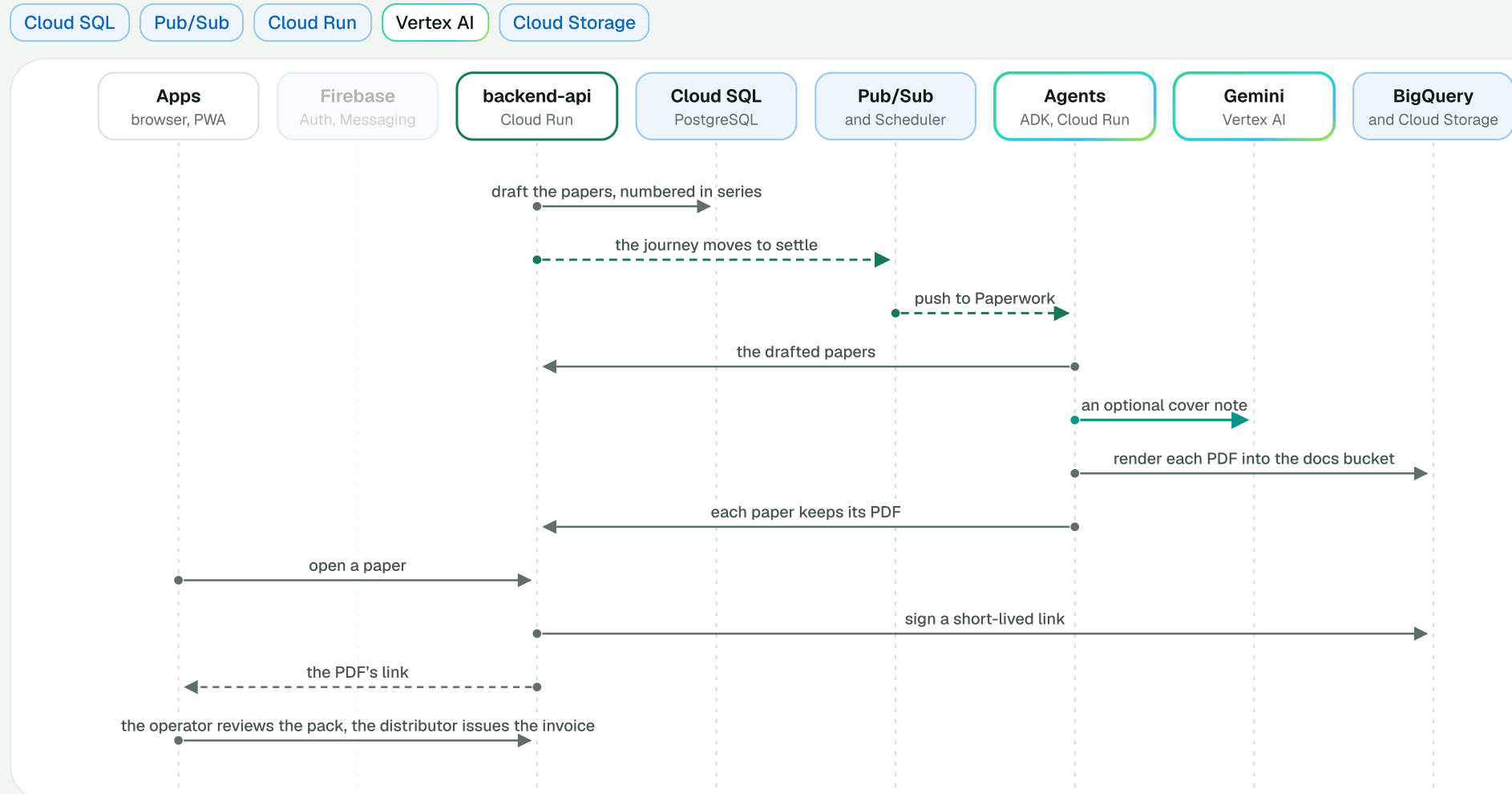
One event starts three agents at once in an ADK ParallelAgent: the Lister writes the marketplace listing, Outreach writes the kiranas' offer in Hindi, English and Marathi, and Donation books the food bank. Kiranas order in the app; a buyer bids, backend-api decides the counter, and the Negotiator words the reply.



The reserve price never reaches a prompt, and a buyer's message is quoted between delimiters as data. A reply to a bid must state the decided price and nothing else.

Settle · Paperwork

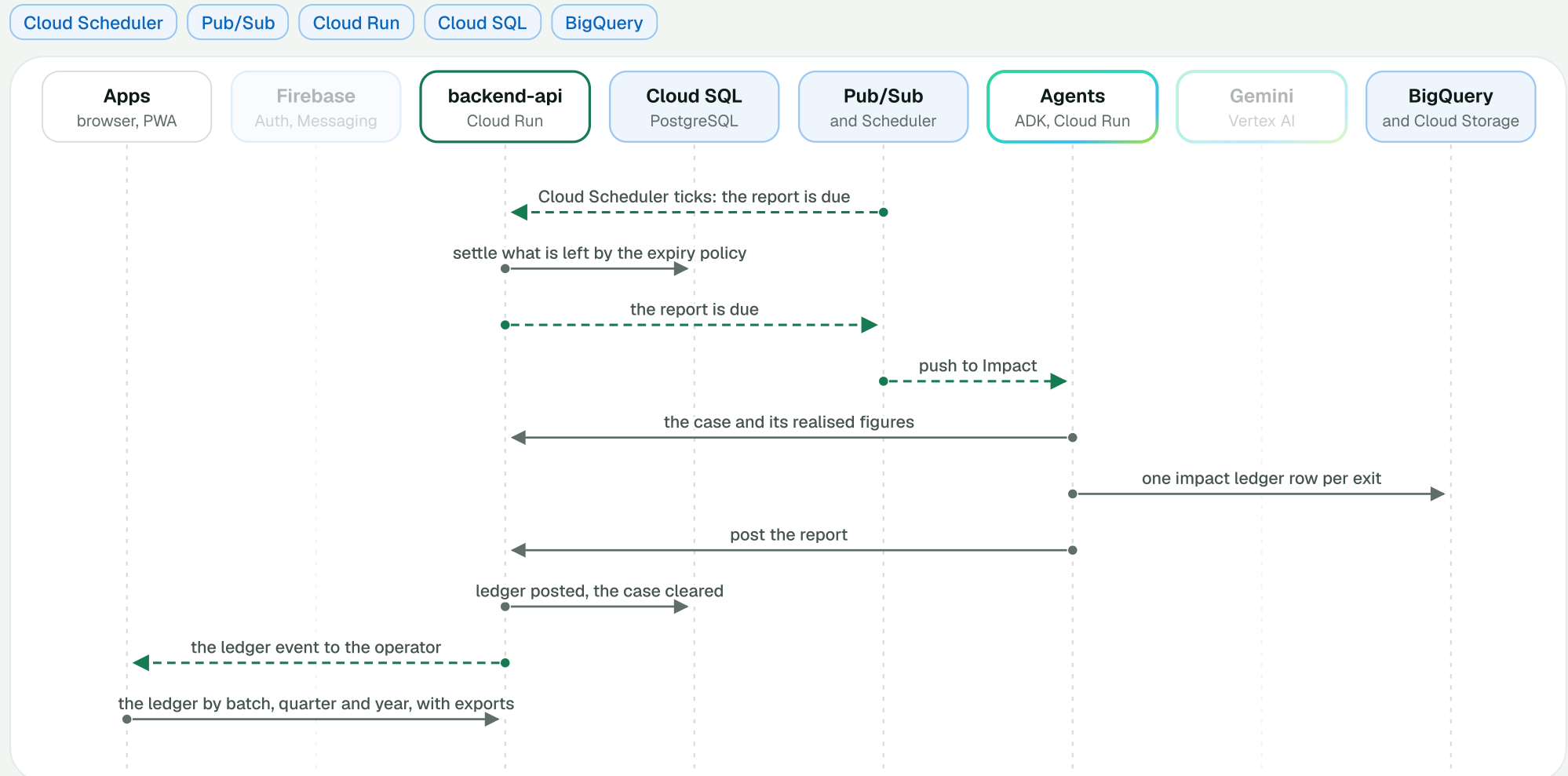
Once every line of the plan is done, backend-api drafts the papers, numbered in their series: the tax invoice, the credit notes, the GST input-credit memo, the food bank's receipt. Paperwork renders each as a PDF with WeasyPrint into Cloud Storage, and members open them through short-lived signed links.



The figures on every paper are backend-api's. Gemini Flash may add a one-line cover note; if it fails, the paper ships without it.

Report · Impact

When the batch's report falls due, or the console calls expiry day, backend-api closes what is left by the client's expiry policy. Impact posts the ledger: what each exit recovered, the input credit kept and reversed, the kilos kept out of landfill, the meals. The rows go to BigQuery's impact table and to the workspace's ledger.



The ledger reads by batch, by quarter of the Indian financial year and by year, and exports the BRSR table, the GST summary and the ledger itself.

Every caller signs as someone, and nobody holds a key

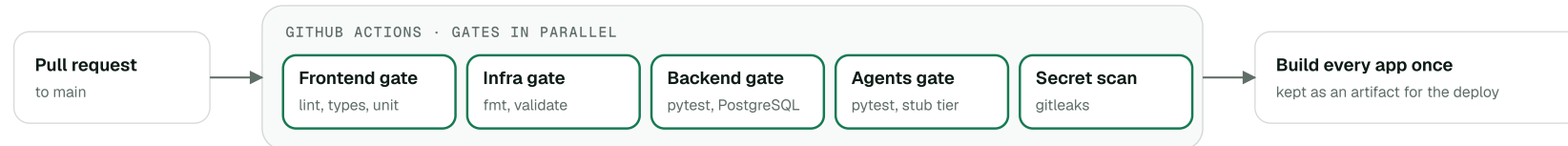
People sign in with Firebase; machines sign with OIDC tokens Google mints for a service account. Each account may do one job, and the database accepts IAM logins only.

WHO ACTS	SIGNS AS	MAY
A member in an app Firebase email and password	Firebase ID token verified by backend-api	See a view cut to their role a buyer never sees the reserve, a kirana sees its own offer, partners never see the brand's margins
Cloud Scheduler, Pub/Sub the clock and the event bus	sc-invoker OIDC token for a fixed audience	Call the tick and the agents Cloud Run's IAM lets nobody else into the agents service
The agents service on Cloud Run	sc-agents Google ID token to backend-api	Read facts, report words Vertex AI, BigQuery, Cloud Storage and /internal; never the database
backend-api and its jobs Cloud Run service and jobs	sc-api, sc-migrator IAM database logins	Write the record data access as the app, schema changes only as the migrator; the audit log refuses edits even to its owner
GitHub Actions on main, environment prod	github-backend, github-deployer Workload Identity Federation	Start Cloud Build as sc-builder, release Hosting nothing else
A developer's laptop their own gcloud login	sc-api-local, sc-agents-local impersonated in code	Run the real stack locally the same Firebase pool, secrets, topics and buckets, on a local PostgreSQL

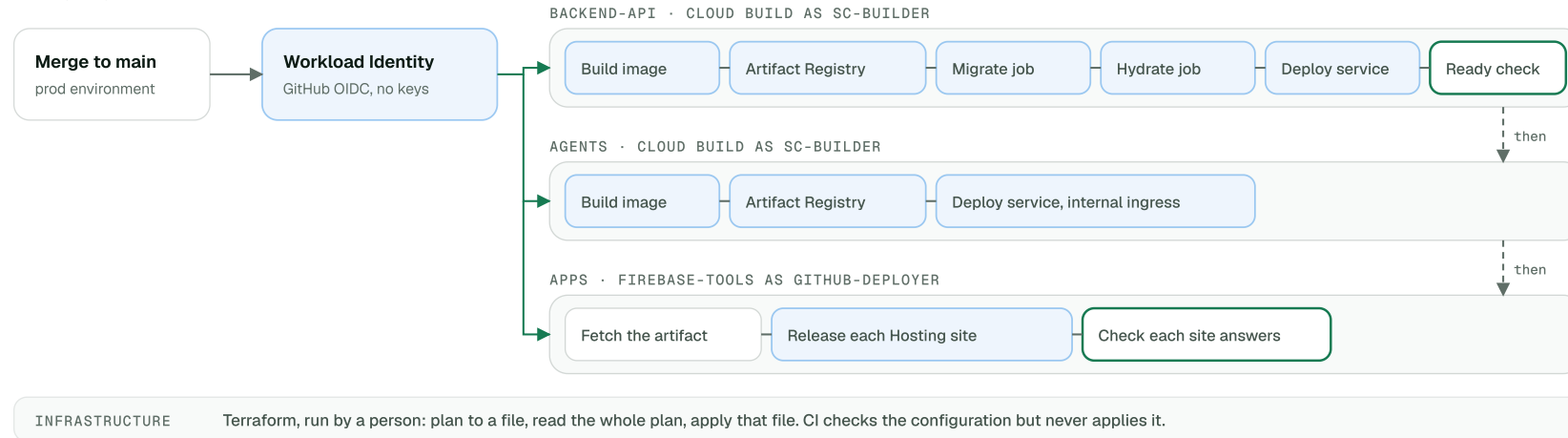
GitHub Actions gates every change; Cloud Build ships it to Cloud Run

A pull request runs five gates in parallel and builds the apps once. A merge to main signs in to Google with Workload Identity Federation, has Cloud Build build, migrate and deploy backend-api and then the agents, and releases the apps' build to Firebase Hosting.

EVERY PULL REQUEST



A MERGE TO MAIN



Jobs wait for the gates they need, and deploys queue instead of overlapping. Every action is pinned to a commit SHA, the workflow token is read-only, and only the deploying jobs may request an identity token.

Path-aware

A merge rebuilds backend-api only when backend-api/ changed and the agents only when agents/ changed; the apps release on every merge that touches the frontend, the designs or the infrastructure.

Migrate before deploy

Cloud Build runs the migration job on the new image before the service moves to it, so the schema is always ahead of the code reading it.

Ordered releases

backend-api deploys first, then the agents that call it, then the apps that read it, because the landing page prerenders against the live API.

Images owned by the build

Cloud Build tags each image with its commit; Terraform owns the service's configuration and ignores the image, so neither undoes the other.

TESTING

Proven at every layer, from a pixel to a live Gemini reply

Fast gates run on every pull request. The browser suites (accessibility, end to end, parity, live and journey) and the live-model evals run when we ask for them, on a real machine, against real builds and real services.

Gates

EVERY PULL REQUEST

[Vitest](#) · [pytest](#) · [ruff](#) ·
[ESLint](#) · [svelte-check](#) ·
[gitleaks](#)

- Frontend: unit tests, drift tests that hold ported CSS to the design, golden texts of every role's screens, seed and icon checks.
- Backend: pytest on a real PostgreSQL service container, the live journey walked over HTTP with in-memory Pub/Sub, Storage and Cloud Messaging.
- Agents: every pipeline on recorded Gemini responses, its exact requests, its fallbacks, and the reserve never reaching a model.
- Contracts: the money rules exist in JavaScript, TypeScript and Python, and shared fixtures hold all three to the same answers.

Accessibility

ON REQUEST

[Playwright](#) · [@axe-core/playwright](#) · [WCAG 2.2 AA](#)

Each app's production build, scanned in desktop, tablet and phone viewports, light and dark, with zero violations. Keyboard specs cover focus in sheets and alerts, the menu-button pattern and sign-in by keyboard; a motion spec fails any endless animation. A coverage check fails the run if an app uses a design-system component that no scan had on screen.

End to end

ON REQUEST

[Playwright](#) · [Chromium](#) ·
[Firefox](#) · [WebKit](#)

The console's flows, the workspace journey on its stub, and the guided demo played beat by beat, on each app's build. Smoke runs in Firefox and in WebKit on iPad and iPhone profiles.

Parity

ON REQUEST

[Playwright](#) · [pixelmatch](#)

Each app's build is compared with its approved design3 prototype, pixel by pixel, at desktop, tablet and phone sizes in both themes. The port cannot drift from the design that was signed off.

Live

ON REQUEST

[Vitest + jsdom](#) · [Playwright](#)
· [Firebase Auth](#)

The workspace's live tests draw its screens on what backend-api actually answered at moments of the journey, recorded from the running API. The live end-to-end suite drives the landing page and the console against the real API and real Firebase sign-in, and reads each change back from the audit log under its own trace.

Journey

ON REQUEST

[Playwright](#) · [Firebase custom tokens](#) · [live Gemini](#)

Munchly Chips, Munchly Mango and their leftover variants. From a console reset to the impact report, every person acts in the real UI in turn: the operator, the distributor, each ordering kirana, the buyer and the food bank, with the agents on live Gemini. Sign-in uses custom tokens minted by an impersonated service account, so no password is handled. Each run records a video, a still per step and a report of findings, against the local stack or production.

Evals

ON REQUEST

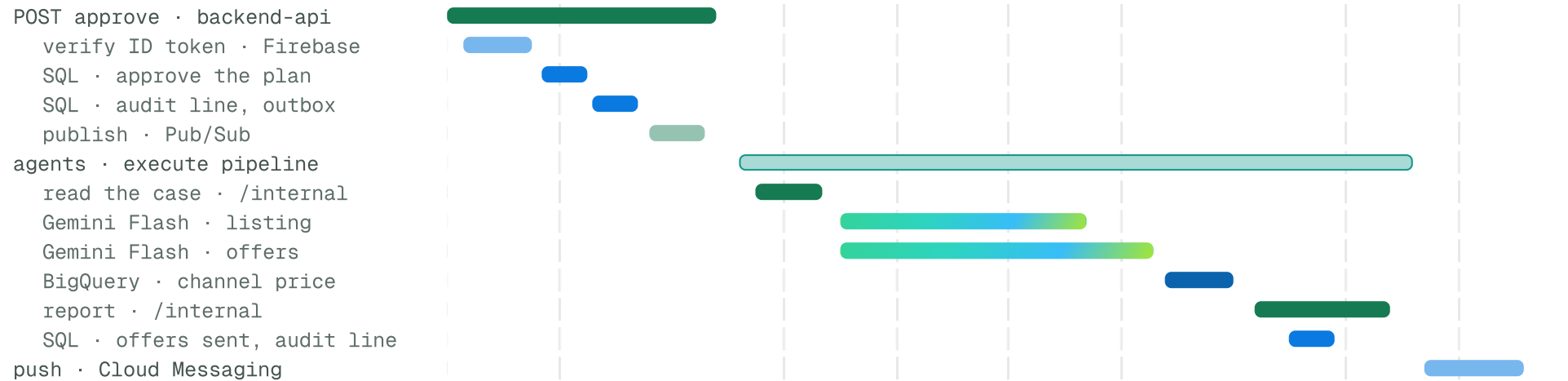
[live Gemini](#) · [deterministic scorers](#) · [Gemini Pro judge](#)

Every agent's eval set runs through the same writer the pipeline calls: label photos rendered with blur, glare and tape for Vision, odd DMS headers for the Data agent, hostile buyer messages for the Negotiator. Scorers check the hard rules, a judge scores the words on a rubric, and results land in BigQuery.

SUITE	DESIGN PROTOTYPE	APP CODE	REAL BROWSER	BACKEND- API	POSTGRESQL	FIREBASE AUTH	PUB/SUB, STORAGE	AGENTS	GEMINI
Frontend gate	●	●							
Backend gate				●	●	◐	◐		
Agents gate				◐			◐	●	◐
Accessibility		●	●						
End to end		●	●						
Parity	●	●	●						
Live		●	●	●	●	●			
Journey		●	●	●	●	●	●	●	●
Evals								●	●
● the real thing ◐ recorded, faked or in memory									

One trace, from a tap to an agent's report

The apps send a traceparent with every call. Cloud Run keeps it, backend-api continues it into its spans, log lines and audit rows, and the outbox hands it to Pub/Sub, so the agents' run and its Gemini calls land in the same trace.



An illustrative trace as Cloud Trace draws it. Bars show order and nesting; their lengths are illustrative.

Logs carry the trace

Every JSON line written during a request names its trace and span, so Logs Explorer shows it under Cloud Run's own request line.

Audit rows carry the trace

Each change's audit row keeps the trace of the request that made it, so a line in the console leads back to its request.

Agent runs carry the trace

Each run's report and its BigQuery row hold its trace id, with model, latency and whether it fell back.

Sampled where it counts

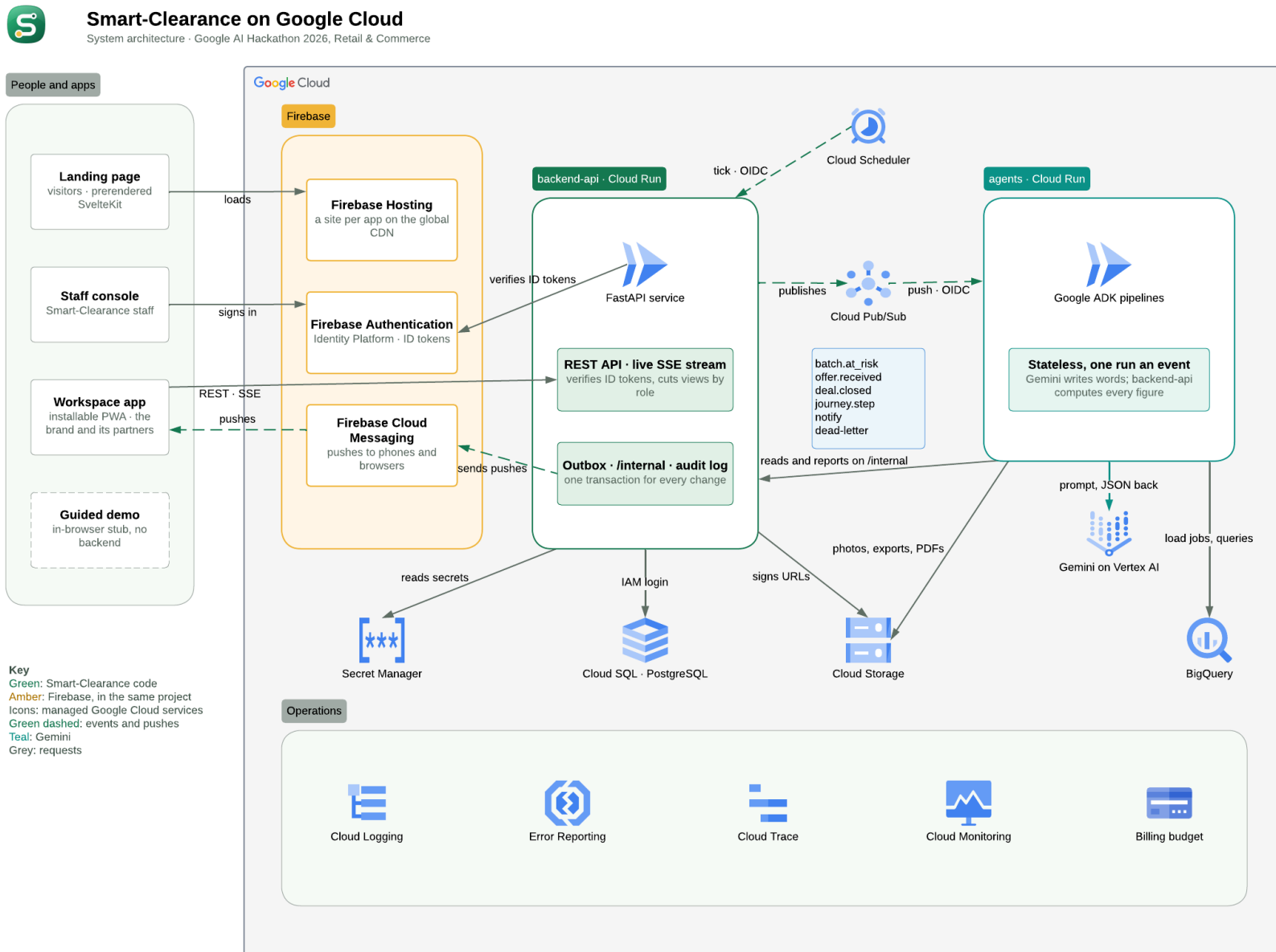
A request Cloud Run sampled is always kept, a share of the rest is sampled, and every request keeps its trace id in its logs regardless.

The diagrams in Lucidchart

The architecture and the application flow, drawn in Lucidchart with Google Cloud's own icon set. The pages that follow are exports of three Lucid documents:

- **Smart-Clearance Architecture on Google Cloud:** the system, the agents' nine stops, CI/CD
<https://lucid.app/lucidchart/0e33214e-b46f-4cfb-9413-b2c40de7d1c6/view>
- **Smart-Clearance Application Flow - the core loop:** the sequence every stop runs on
<https://lucid.app/lucidchart/2965aa52-3190-4d68-8791-9c95c8fd7bbc/view>
- **Smart-Clearance Application Flow - a batch's journey:** every stop, request by request
<https://lucid.app/lucidchart/c2efb94b-7aa2-4cc1-a6cc-af4980ff255f/view>

System architecture

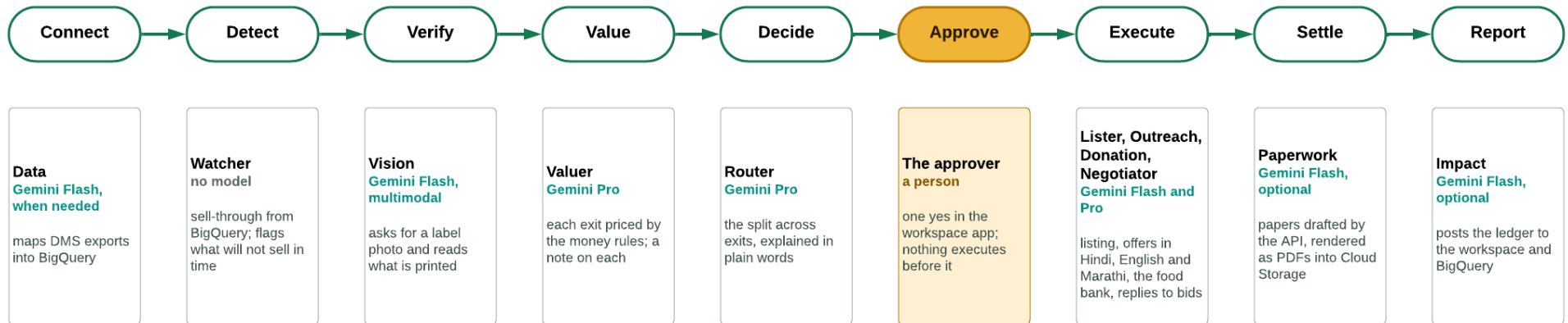


The agents' nine stops

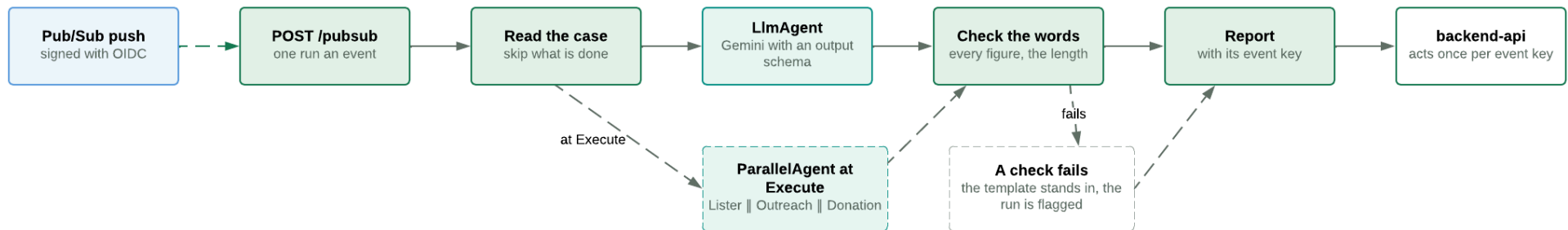


Ten agents along nine stops, and one person's yes

ADK pipelines on Cloud Run · Gemini writes the words, backend-api computes every figure



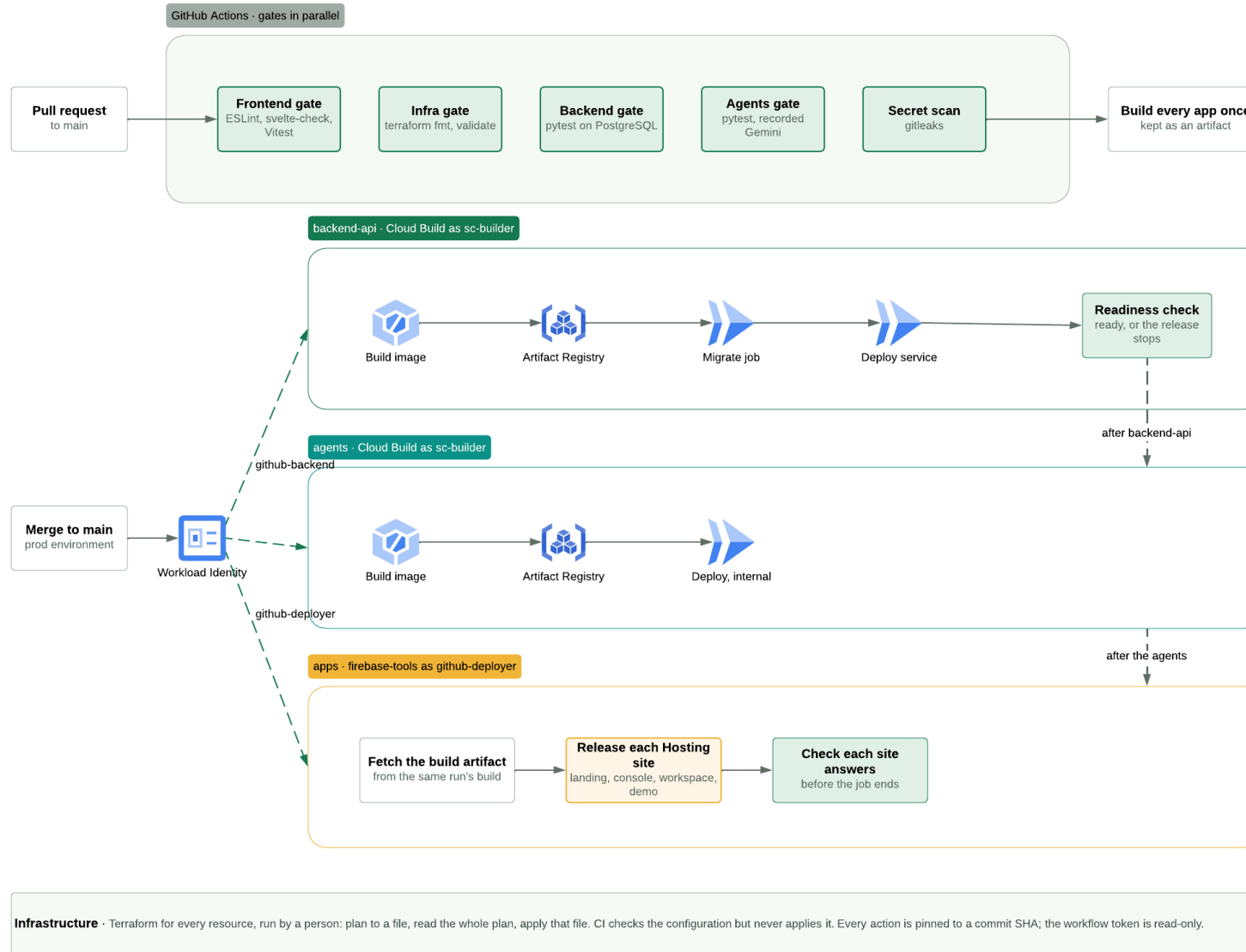
Inside one agent run · an ADK SequentialAgent, one run an event



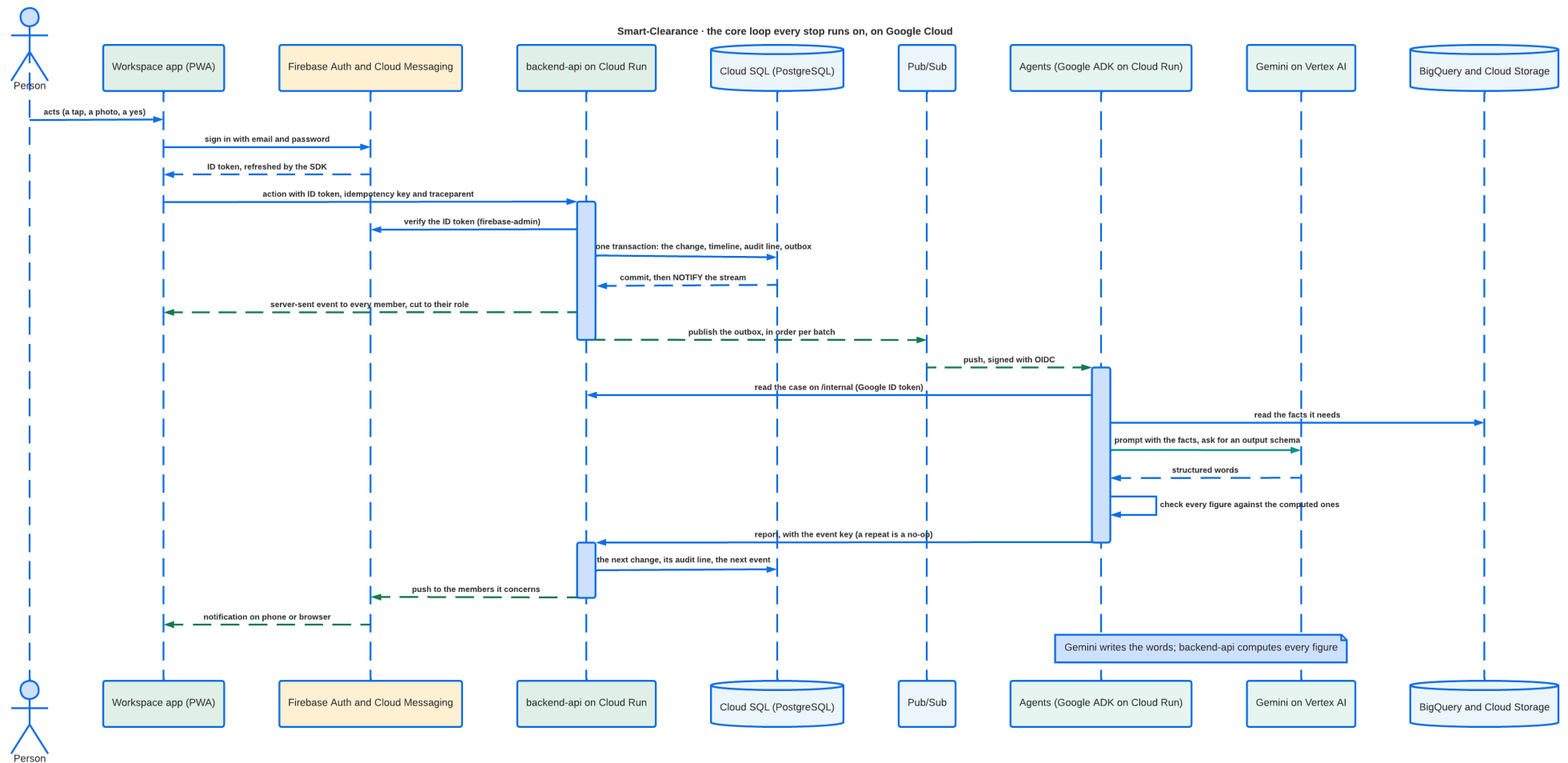


GitHub Actions gates every change; Cloud Build ships it to Cloud Run

Keyless: Workload Identity Federation from the protected prod environment on main

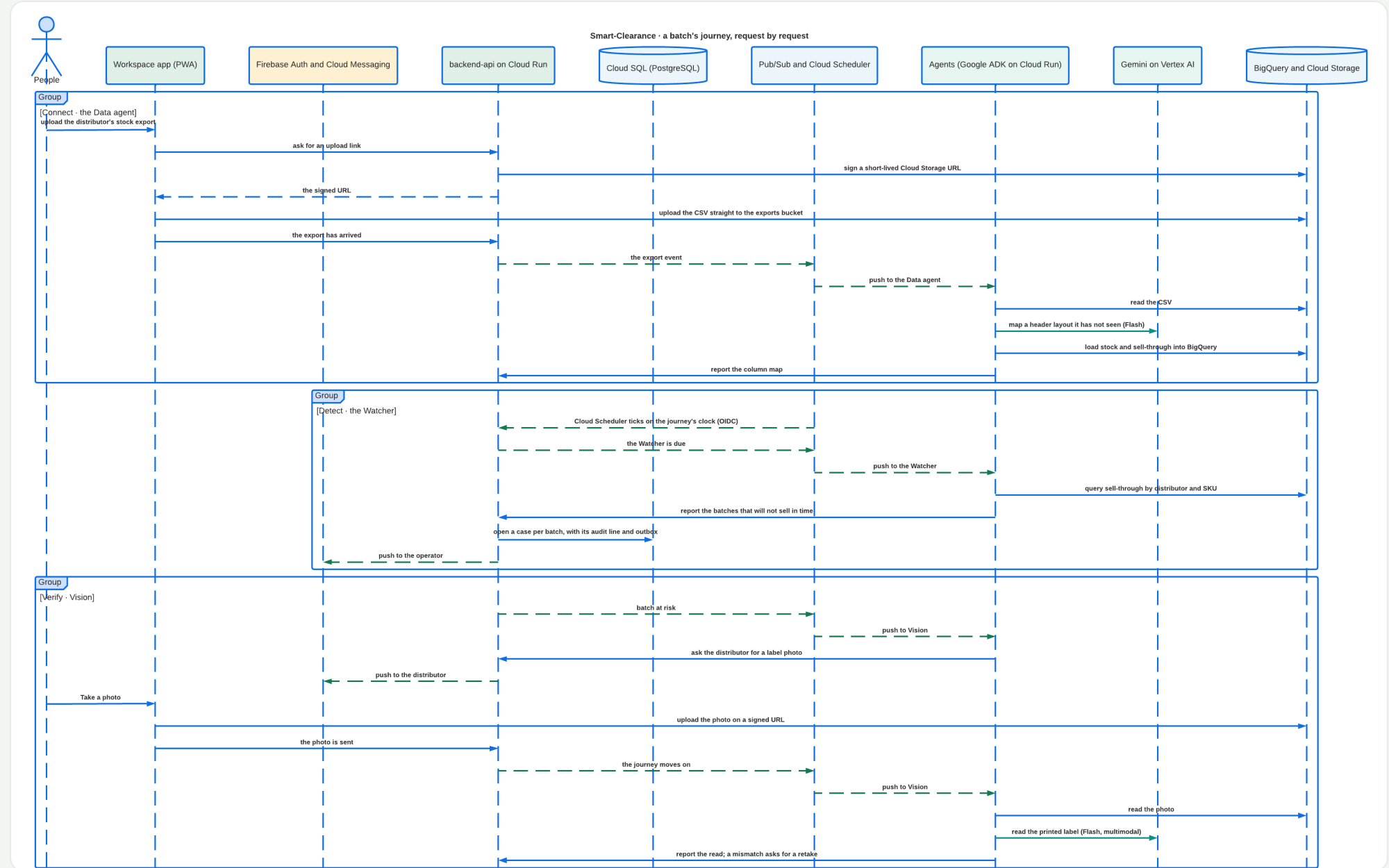


Application flow · the core loop

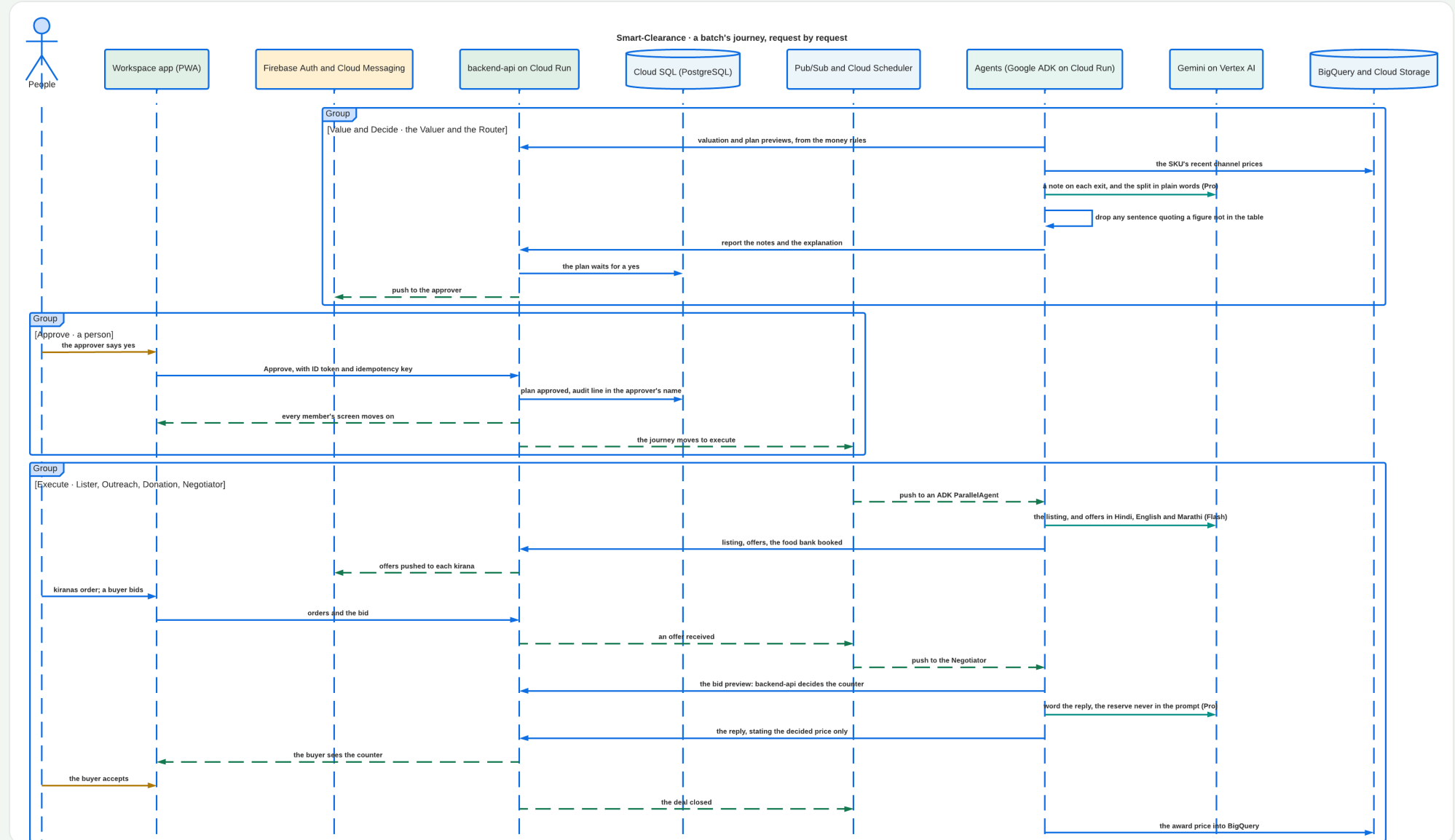


Lucidchart · Smart-Clearance Application Flow · the core loop · <https://lucid.app/lucidchart/2965aa52-3190-4d68-8791-9c95c8fd7bbc/view>

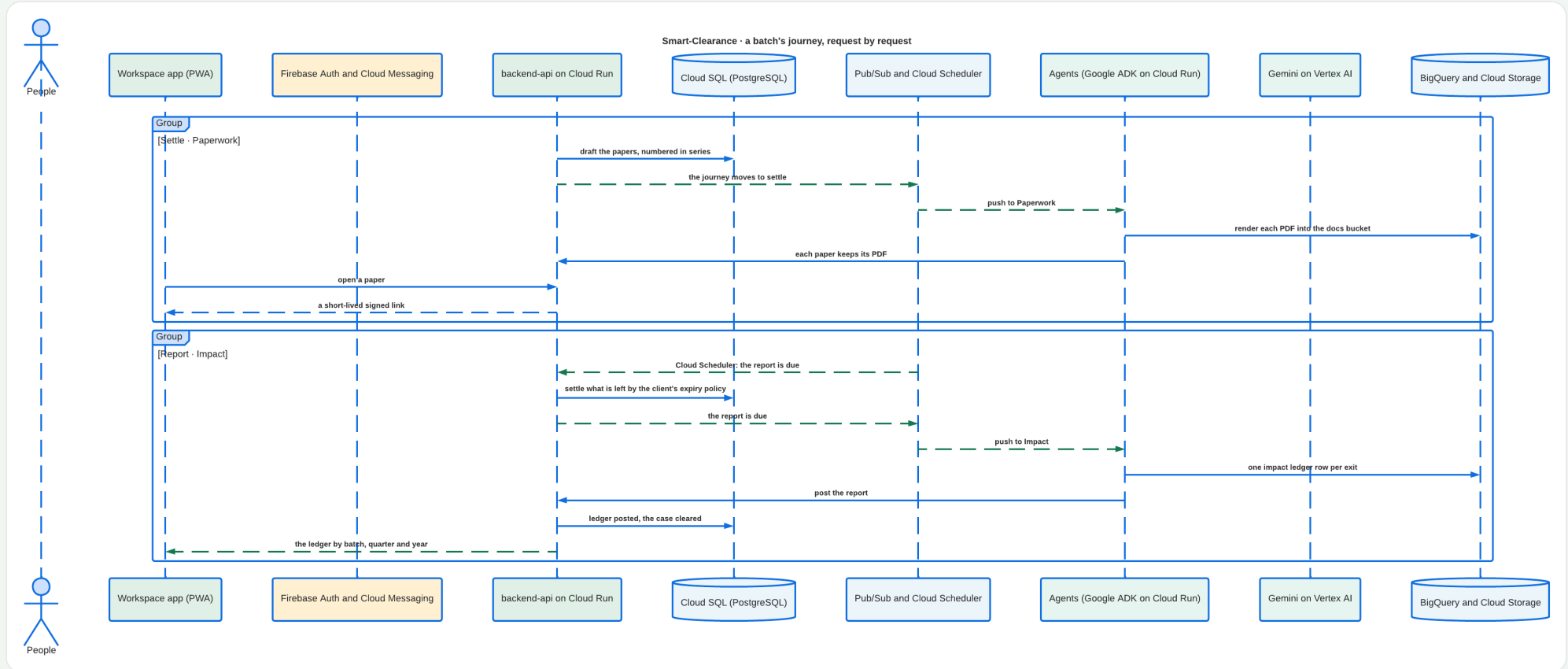
Application flow · a batch's journey: Connect, Detect, Verify



Application flow · a batch's journey: Value and Decide, Approve, Execute



Application flow · a batch's journey: Settle, Report



Lucidchart · Smart-Clearance Application Flow · a batch's journey, part three of three · <https://lucid.app/lucidchart/c2efb94b-7aa2-4cc1-a6cc-af4980ff255f/view>